bienvenida

Mientras esperas...

Si lo dejas hecho, empezamos todos a la vez.

1 Instala Git

`git-scm.com/downloads`

2 Crea tu cuenta

`github.com/signup`

3 Prepara VS Code

Inicia sesión con GitHub desde el icono de cuentas, abajo a la izquierda.

4 Di quién eres

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email"
```

Taller de iniciación

Git y GitHub



AISC x GUL UC3M

slides

Las diapositivas

Por si quieres seguirlas desde el portátil.



PENDIENTE: falta el enlace de este año

formato

Formato del taller

Lo básico de Git, y practicando desde el principio.

Git

`git-scm.com`

GitHub

`github.com`

VS Code

`code.visualstudio.com`



Cuando salga este icono, **toca practicar**. Si te pierdes, levanta la mano.

La idea

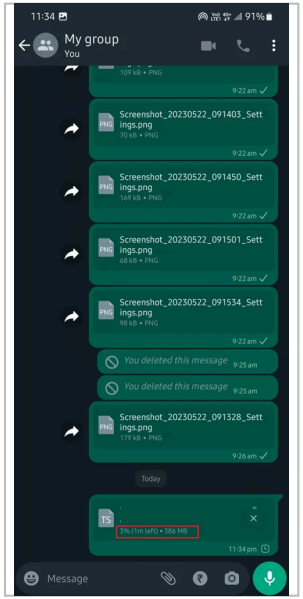
01


La idea
Empezar
Guardar
Dónde estás
Traer cambios
En paralelo

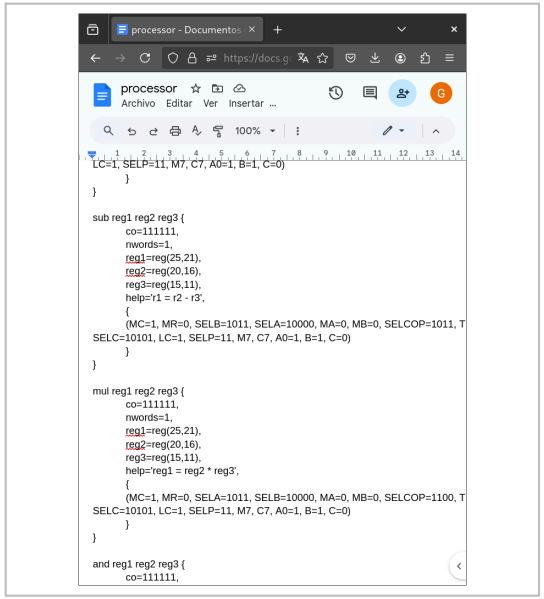
por-que-git

¿Por qué Git?

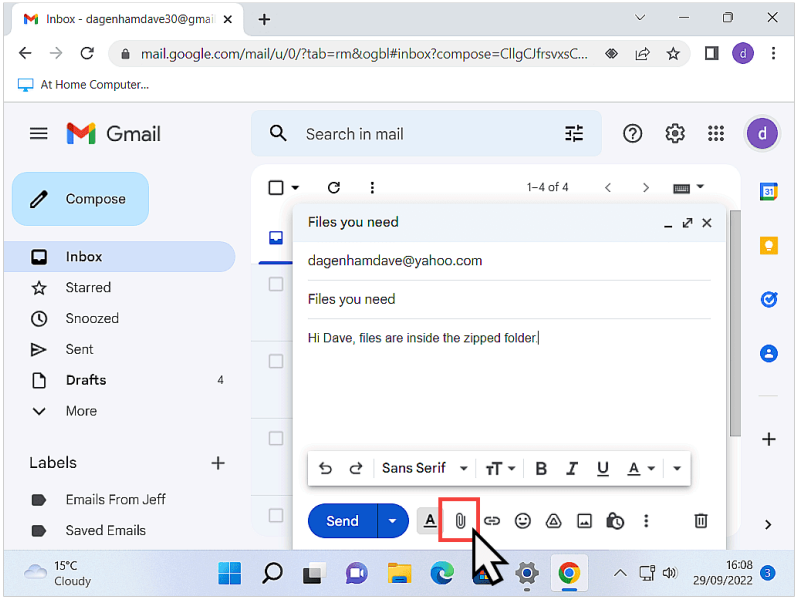
Porque pasarse el código por aquí siempre acaba en lío.



WhatsApp



Drive



Gmail

git-y-github

Git no es GitHub

Van de la mano, pero no son lo mismo.



Git

el programa, en tu ordenador

Guarda el historial de tu proyecto.

Funciona sin internet.

Linus Torvalds lo creó en 2005 para el kernel de Linux.



GitHub

la web donde lo subes

Guarda una copia en internet.

Es donde trabajas con los demás.

Hay alternativas: GitLab, Forgejo, Gitea.

Más de 14.000 personas han tocado el kernel de Linux trabajando así.





zonas

Las cuatro zonas

En Git, casi todo es mover cambios de una a otra.



En cada diapositiva, arriba, verás en azul en qué zona actúa el comando:

Trabajo › Stage › **Local** › Remoto

Empezar un repo

`init · clone`

SOURCE CONTROL

Initialize Repository

Te sale en Source Control si la carpeta no tiene repo.

git init

Trabajo › Stage › **Local** › Remoto

Empezar un repo desde cero

ANTES Una carpeta normal. Git ni se entera.



TERMINAL

```
$ git init
```

`-b <rama>` elige el nombre de la rama inicial

Cuidado
No lo hagas dentro de otro repo.

Deshacer
Borra la carpeta `.git`.
Tus archivos no se tocan.

SOURCE CONTROL

Initialize Repository

Te sale en Source Control si la carpeta no tiene repo.

git init

Trabajo › Stage › **Local** › Remoto

Empezar un repo desde cero

DESPUÉS Aparece la carpeta oculta `.git`, que es donde se guarda todo.



TERMINAL

```
$ git init
```

`-b <rama>` elige el nombre de la rama inicial

Cuidado
No lo hagas dentro de otro repo.

Deshacer
Borra la carpeta `.git`.
Tus archivos no se tocan.

SOURCE CONTROL

Clone Repository

Te sale en Source Control si no tienes ninguna carpeta abierta.

git clone

Trabajo › Stage › **Local** › Remoto

Traer un repo de GitHub

ANTES El proyecto solo está en GitHub.



TERMINAL

```
$ git clone <url>
```

`-b <rama>`	te deja ya en esa rama
`--depth 1`	se baja solo el último commit

⚠ Cuidado
Crea una carpeta nueva. Acuérdate de entrar en ella.

Deshacer
Borra la carpeta.
En GitHub no cambia nada.

SOURCE CONTROL

Clone Repository

Te sale en Source Control si no tienes ninguna carpeta abierta.

git clone

Trabajo › Stage › **Local** › Remoto

Traer un repo de GitHub

DESPUÉS Te bajas los archivos y todo el historial, no solo la última versión.



TERMINAL

```
$ git clone <url>
```

`-b <rama>`	te deja ya en esa rama
`--depth 1`	se baja solo el último commit

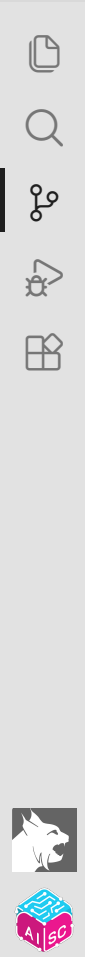
⚠ Cuidado
Crea una carpeta nueva. Acuérdate de entrar en ella.

Deshacer
Borra la carpeta.
En GitHub no cambia nada.

Guardar el trabajo

`diff` · `add` · `commit` · `status` · `log` · `push`





SOURCE CONTROL

CHANGES 1

README.md M

Pincha en el archivo y ves los cambios.

git diff

Trabajo › Stage › Local › Remoto

Ver qué has cambiado

EJEMPLO En rojo lo que quitas y en verde lo que añades.

README.md

```
  # Mario Bros
- Un juego de plataformas.
+ Un juego de plataformas en Python.
+ Para jugar: python mario.py
  Hecho en el taller de Git.
```

GRAPH

● Primera versión `main` `origin`

TERMINAL

```
$ git diff [<archivo>]
```

`--staged`	lo que ya tienes en Stage
`--stat`	solo un resumen por archivo

⚠ Cuidado

Lo que ya hayas metido con add no sale sin `--staged`.

SOURCE CONTROL

CHANGES 3

salto.py + U

fuego.py U

notas.txt U

El + de cada archivo hace lo mismo que `git add`.

GRAPH

Primera versión main origin

git add

Trabajo › **Stage** › Local › Remoto

Preparar cambios

ANTES Tres archivos nuevos y ninguno preparado.



TERMINAL

```
$ git add <archivo>
```

`.`	todo lo que hay en esta carpeta
`-p`	vas eligiendo trozo a trozo

⚠ Cuidado

`git add .` lo mete todo. Mira antes `git status`.

Deshacer

`git restore --staged <archivo>`
lo saca de Stage y no se pierde.

SOURCE CONTROL

CHANGES 3

salto.py + U

fuego.py U

notas.txt U

El + de cada archivo hace lo mismo que `git add`.

GRAPH

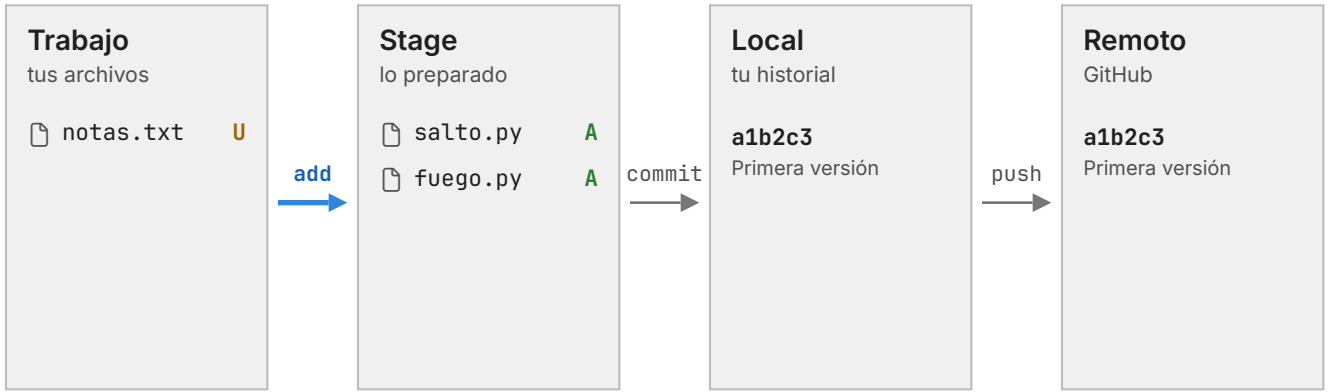
Primera versión main origin

git add

Trabajo › Stage › Local › Remoto

Preparar cambios

DESPUÉS Con `git add salto.py fuego.py` eliges qué va en el próximo commit.



TERMINAL

```
$ git add <archivo>
```

- `.` todo lo que hay en esta carpeta
- `-p` vas eligiendo trozo a trozo

Cuidado

`git add .` lo mete todo. Mira antes `git status`.

Deshacer

`git restore --staged <archivo>`
lo saca de Stage y no se pierde.

SOURCE CONTROL

CHANGES 2

.gitignore U

notas.txt U

.venv/ y __pycache__/ ya no salen en la lista.

.gitignore

Trabajo › Stage › Local › Remoto

Que Git pase de ciertos archivos

EJEMPLO Lo que esté en la lista deja de salir como cambio.

```
.gitignore
1  .venv/
2  __pycache__/
3  .vscode/
4  .idea/
```



Tu carpeta

.gitignore	U
notas.txt	U
.venv/	ignorado
__pycache__/	ignorado

GRAPH

Primera versión main origin

PATRONES

.venv/	el entorno virtual de Python
__pycache__/	la basura que genera Python
.vscode/	los ajustes de VS Code
.idea/	los ajustes de PyCharm

Cuidado

Lo ya commiteado se queda. Para sacarlo:
`git rm --cached <archivo>`.

Deshacer

Quita la línea de `.gitignore`.

SOURCE CONTROL

Mario salta

Commit

STAGED CHANGES 2

salto.py A

fuego.py A

CHANGES 1

notas.txt U

GRAPH

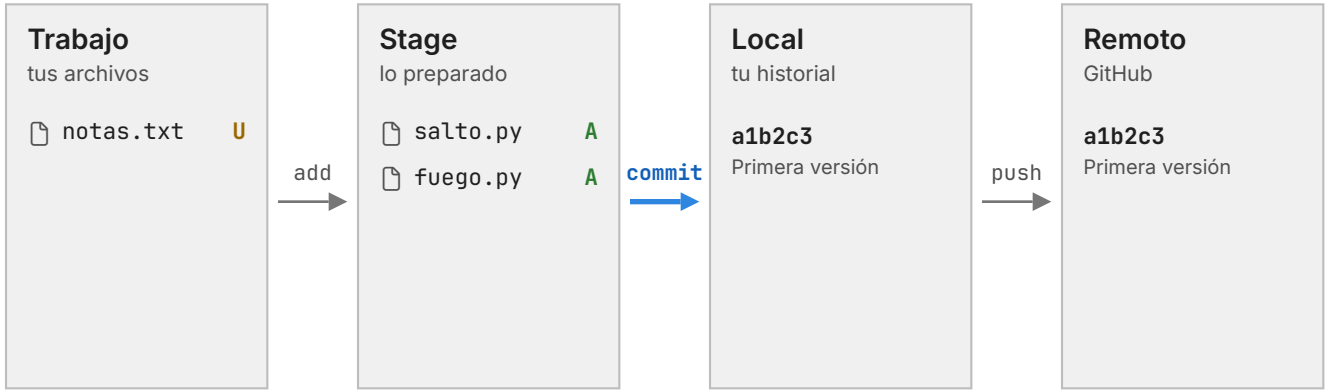
Primera versión main origin

git commit

Trabajo › Stage › **Local** › Remoto

Guardar una versión

ANTES Tienes dos archivos preparados en Stage.



TERMINAL

```
$ git commit -m "<mensaje>"
```

`-a`	añade lo modificado (no lo nuevo)
`--amend`	corrige el último commit

Cuidado

«fix» no dice nada. Pon qué has cambiado.

Deshacer

`git reset --soft HEAD~1`
y no pierdes los cambios.

SOURCE CONTROL

Mario salta

Commit

STAGED CHANGES 2

salto.py A

fuego.py A

CHANGES 1

notas.txt U

GRAPH

Mario salta main

Primera versión origin

git commit

Trabajo › Stage › **Local** › Remoto

Guardar una versión

DESPUÉS Lo que había en Stage pasa a ser una versión nueva. `notas.txt` se queda fuera.



TERMINAL

```
$ git commit -m "<mensaje>"
```

`-a`	añade lo modificado (no lo nuevo)
`--amend`	corrige el último commit

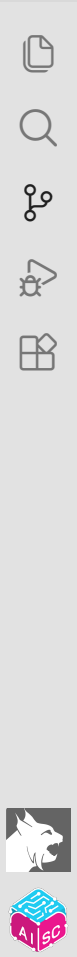
Cuidado

«fix» no dice nada. Pon qué has cambiado.

Deshacer

```
git reset --soft HEAD~1
```

y no pierdes los cambios.



SOURCE CONTROL

STAGED CHANGES 1

.gitignore A

CHANGES 2

README.md M

notas.txt U

Este panel es básicamente `git status`, siempre a la vista.

GRAPH

Mario salta main

Primera versión origin

git status

Trabajo › Stage › Local › Remoto

Ver cómo va todo

EJEMPLO Es lo que te sale, quitando las líneas de ayuda.

```
On branch main                          — Tu rama
Changes to be committed:
    new file:   .gitignore              — En Stage: va en el próximo commit
Changes not staged for commit:
    modified:   README.md               — Cambiado, pero sin preparar
Untracked files:
    notas.txt                           — Archivo nuevo: Git aún no lo sigue
```

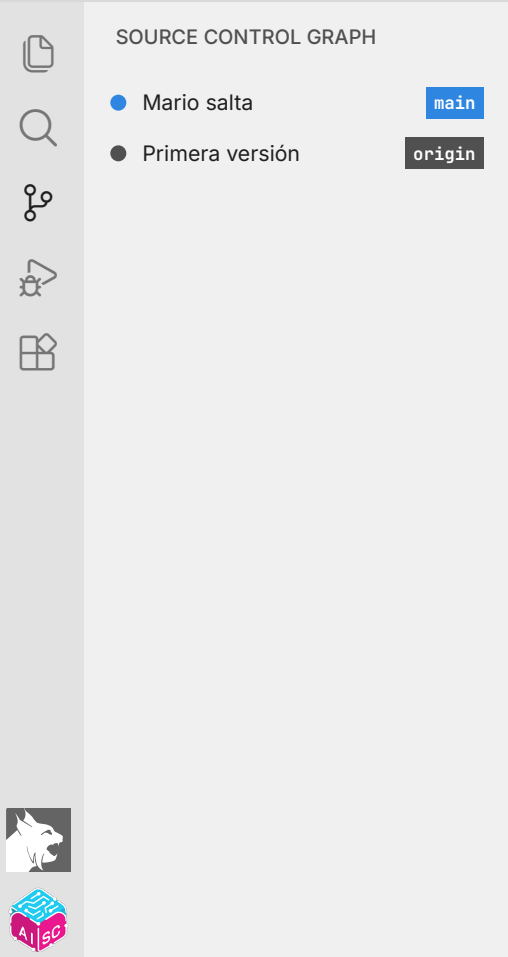
TERMINAL

$ git status

`-s`	versión corta
`-sb`	corta y con la rama

⚠ Cuidado

¿No sale lo que esperabas?
Mira si estás en la carpeta buena.



git log

Trabajo › Stage › **Local** › Remoto

Ver el historial

EJEMPLO Esto es `git log --oneline`, con lo más nuevo arriba.

```
d4e5f6a (HEAD → main) Mario salta        — Estás aquí, en tu último commit

a1b2c3d (origin/main) Primera versión    — GitHub llega hasta aquí
```

TERMINAL

$ git log

`--oneline`	una línea por commit
`--graph`	dibuja las ramas
`-p`	con los cambios de cada commit

⚠ Cuidado
Si es largo se queda abierto.
Para salir, pulsa q.

SOURCE CONTROL

Sync Changes

Hace `pull` y luego `push`.
La primera vez sale Publish Branch.

`git push`

Trabajo › Stage › Local › **Remoto**

Subir los commits

ANTES Tu commit todavía está solo en tu ordenador.



GRAPH

Mario salta `main`
Primera versión `origin`

TERMINAL

```
$ git push origin <rama>
```

`-u`	recuerda el remoto de esta rama
`--force`	machaca lo de GitHub con lo tuyo
`--force-with-lease`	igual, pero solo si nadie ha subido nada

⚠ Cuidado
`--force` se puede cargar el trabajo de los demás.

Deshacer
`git revert <commit>` y `push`.
Crea un commit que lo deshace.

SOURCE CONTROL

Sync Changes

Hace `pull` y luego `push`.
La primera vez sale Publish Branch.

`git push`

Trabajo › Stage › Local › **Remoto**

Subir los commits

DESPUÉS Ya está en GitHub y los demás se lo pueden bajar.



GRAPH

Mario salta `main` `origin`
Primera versión

TERMINAL

```
$ git push origin <rama>
```

`-u`	recuerda el remoto de esta rama
`--force`	machaca lo de GitHub con lo tuyo
`--force-with-lease`	igual, pero solo si nadie ha subido nada

⚠ Cuidado
`--force` se puede cargar el trabajo de los demás.

Deshacer
`git revert <commit>` y `push`.
Crea un commit que lo deshace.

SOURCE CONTROL

Sync Changes

Hace `pull` y luego `push`.
La primera vez sale Publish Branch.

`git push`

Trabajo › Stage › Local › **Remoto**

Subir los commits

--FORCE GitHub tenía un commit que tú no tenías, y `--force` se lo carga.



TERMINAL

```
$ git push origin <rama>
```

`-u`	recuerda el remoto de esta rama
`--force`	machaca lo de GitHub con lo tuyo
`--force-with-lease`	igual, pero solo si nadie ha subido nada

⚠ Cuidado
`--force` se puede cargar el trabajo de los demás.

↺ Deshacer
`git revert <commit>` y `push`.
Crea un commit que lo deshace.

Saber dónde estás

`HEAD · origin`

EN VS CODE

main

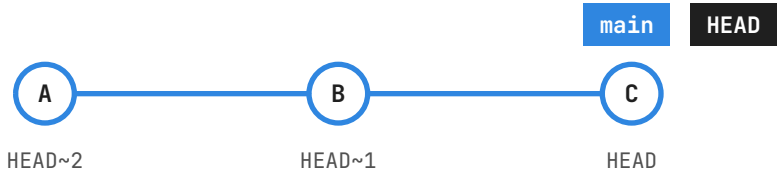
Abajo a la izquierda siempre ves en qué rama estás. Ahí apunta HEAD.

HEAD

Trabajo › Stage › **Local** › Remoto

HEAD: dónde estás ahora

AHORA HEAD apunta a tu rama, y tu rama a su último commit.



TERMINAL

```
$ git show HEAD~<n>
```

HEAD	tu commit actual
HEAD~1	el anterior
HEAD~2	dos más atrás

⚠ Cuidado

Si te sale «detached HEAD», vuelve con `git switch main`.

EN VS CODE

main

Abajo a la izquierda siempre ves en qué rama estás. Ahí apunta HEAD.

HEAD

Trabajo › Stage › **Local** › Remoto

HEAD: dónde estás ahora

TRAS UN COMMIT El commit nuevo mueve `main`, y `HEAD` se va con ella.



TERMINAL

```
$ git show HEAD~<n>
```

`HEAD`	tu commit actual
`HEAD~1`	el anterior
`HEAD~2`	dos más atrás

⚠ Cuidado

Si te sale «detached HEAD», vuelve con `git switch main`.

EN VS CODE

1 Abre la paleta

Ctrl Shift P

2 Escribe el comando

```
>Git: Add Remote...
```

`git clone` ya te crea `origin`.
Solo hace falta tras `git init`.

origin

origin: el mote de tu remoto

AL DÍA `origin` es un mote para esa URL. `origin/main` es por dónde iba GitHub.



TERMINAL

```
$ git remote add origin <url>
```

`-v`	con git remote -v, ves tus remotos
`origin/main`	main en GitHub, la última vez que miraste

⚠ Cuidado

`origin/main` no se actualiza solo.
Hace falta `fetch`, `pull` o `push`.

↺ Deshacer

`git remote remove origin`
solo borra el mote.

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

```
>Git: Add Remote...
```

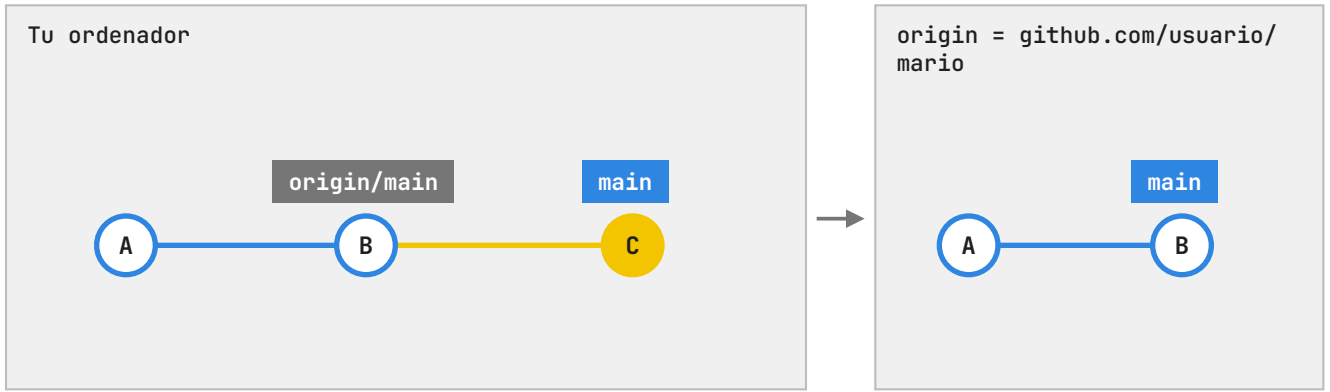
`git clone` ya te crea `origin`.
Solo hace falta tras `git init`.

origin

Trabajo › Stage › Local › **Remoto**

origin: el mote de tu remoto

TRAS UN COMMIT Tu `main` avanza y `origin/main` se queda: GitHub aún no se ha enterado.



TERMINAL

```
$ git remote add origin <url>
```

`-v`	con git remote -v, ves tus remotos
`origin/main`	main en GitHub, la última vez que miraste

⚠ Cuidado
`origin/main` no se actualiza solo.
Hace falta `fetch`, `pull` o `push`.

↺ Deshacer
`git remote remove origin`
solo borra el mote.

Traer cambios

05

`fetch · pull`



La idea · Empezar · Guardar · Dónde estás · **Traer cambios** · En paralelo

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

```
>Git: Fetch
```

También en Source Control:
··· › Pull, Push › Fetch

GRAPH

Mario salta `main` `origin`

Primera versión

git fetch

Trabajo › Stage › **Local** › Remoto

Mirar qué hay nuevo en GitHub

ANTES Un compañero ha subido un commit que tú no tienes.



TERMINAL

```
$ git fetch [<remoto>]
```

`--all`	de todos los remotos
`--prune`	olvida las ramas borradas en GitHub

⚠ Cuidado
No te cambia los archivos.
Para eso está `pull`.

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

```
>Git: Fetch
```

También en Source Control:
··· › Pull, Push › Fetch

git fetch

Trabajo › Stage › **Local** › Remoto

Mirar qué hay nuevo en GitHub

DESPUÉS Ya tienes el commit en `origin/main`, pero tus archivos siguen igual.



GRAPH

- **Añade enemigos** `origin`
- Mario salta `main`
- Primera versión

TERMINAL

```
$ git fetch [<remoto>]
```

`--all`	de todos los remotos
`--prune`	olvida las ramas borradas en GitHub

⚠ **Cuidado**
No te cambia los archivos.
Para eso está `pull`.

SOURCE CONTROL

Sync Changes

Hace `pull` y luego `push`.
Solo traer: ··· › Pull, Push › Pull

`git pull`

Trabajo › Stage › Local › Remoto

Traer los cambios de GitHub

ANTES Tu carpeta se ha quedado por detrás de GitHub.



GRAPH

Mario salta `main` `origin`
Primera versión

TERMINAL

```
$ git pull
```

`--rebase`	pone tus commits encima, sin merge
`--ff-only`	falla si hay que hacer merge

Cuidado
Haz commit antes. Si tienes cambios sin guardar, igual no te deja.

Deshacer
`git reset --hard ORIG_HEAD` borra lo no commiteado.

SOURCE CONTROL

Sync Changes

Hace `pull` y luego `push`.
Solo traer: ··· › Pull, Push › Pull

GRAPH

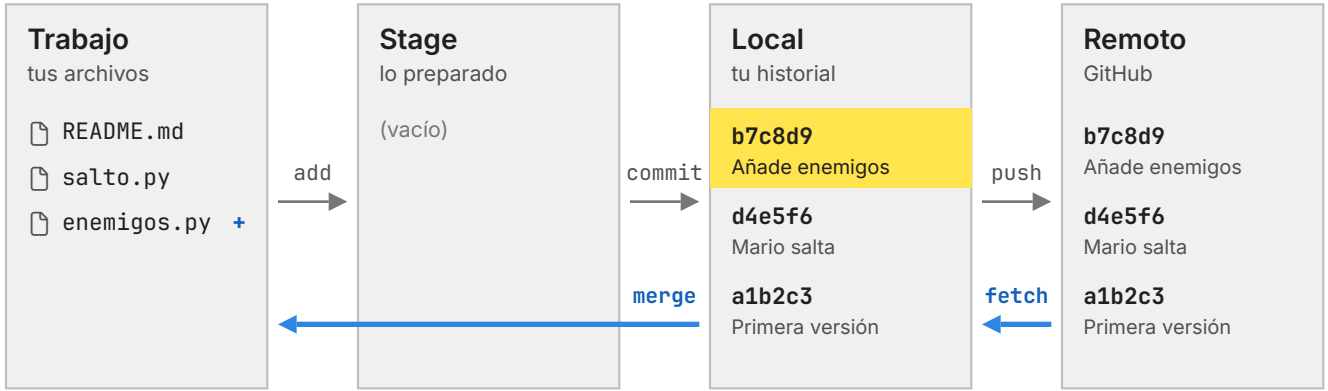
- **Añade enemigos** `main` `origin`
- Mario salta
- Primera versión

`git pull`

Trabajo › Stage › Local › Remoto

Traer los cambios de GitHub

DESPUÉS `pull` es un `fetch` y luego un `merge`, así que ahora sí cambian tus archivos.



TERMINAL

```
$ git pull
```

`--rebase`	pone tus commits encima, sin merge
`--ff-only`	falla si hay que hacer merge

⚠ Cuidado
Haz commit antes. Si tienes cambios sin guardar, igual no te deja.

Deshacer
`git reset --hard ORIG_HEAD` borra lo no commiteado.

ejercicio-1

Ejercicio 1: tu primer repo

Unos 10 minutos, a tu ritmo.



1	Crea un repo en GitHub con README y clónalo	`git clone <url>`
2	Crea `hola.txt` con una línea y mira qué pasa	`git status`
3	Prepáralo y guárdalo	`git add hola.txt; git commit -m "Añade hola"`
4	Súbelo y míralo en la web	`git push`
5	Cambia la línea, mira el cambio y súbelo	`git diff; git commit -am "Cambia hola"; git push`
6	Edita el README en la web y bájatelo	`git pull`

> **Al terminar** En `git log --oneline` y en GitHub salen los mismos commits.

Trabajar en paralelo

branch · switch · merge · conflictos



La idea · Empezar · Guardar · Dónde estás · Traer cambios · **En paralelo**

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

```
>Git: Create Branch...
```

3 Ponle nombre

```
salto
```

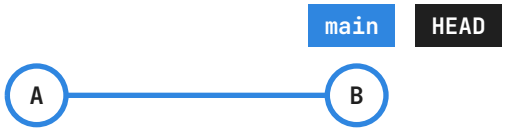
Ojo: aquí además te cambia a la rama nueva.

git branch

Trabajo › Stage › **Local** › Remoto

Crear una rama

ANTES Solo existe `main`.



TERMINAL

```
$ git branch <rama>
```

`-a`	lista todas, también las remotas
`-d <rama>`	borra una rama ya mergeada
`-m <nombre>`	renombra la rama actual

⚠ Cuidado

Crea la rama, pero no te mueve a ella. Falta el `switch`.

Deshacer

`git branch -d <rama>`
la borra, y los commits se quedan.

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

`>Git: Create Branch...`

3 Ponle nombre

`salto`

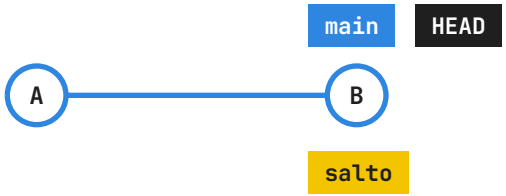
Ojo: aquí además te cambia a la rama nueva.

git branch

Trabajo › Stage › **Local** › Remoto

Crear una rama

DESPUÉS Una rama no es más que una etiqueta sobre un commit. Sigues en `main`.



TERMINAL

```
$ git branch <rama>
```

`-a`	lista todas, también las remotas
`-d <rama>`	borra una rama ya mergeada
`-m <nombre>`	renombra la rama actual

⚠ Cuidado

Crea la rama, pero no te mueve a ella. Falta el `switch`.

Deshacer

`git branch -d <rama>`
la borra, y los commits se quedan.

EN VS CODE

main

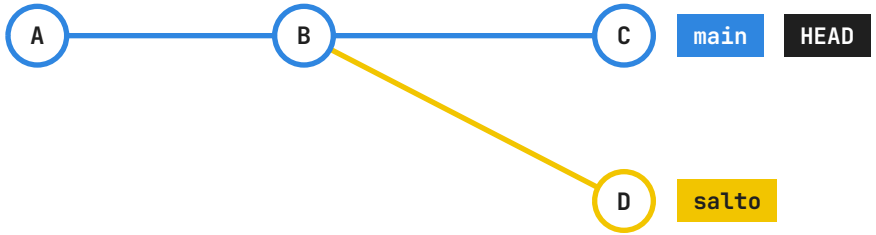
Pincha en tu rama, en la barra de abajo, y elige otra.
Por internet verás `git checkout`, que también vale.

git switch

Trabajo › Stage › Local › Remoto

Cambiar de rama

ANTES Estás en `main`, así que ves los archivos de `C`.



TERMINAL

```
$ git switch <rama>
```

`-c <rama>`	la crea y te mueve a ella
`-`	vuelve a la rama anterior

Cuidado
Si tienes cambios sin guardar, igual no te deja. Haz commit antes.

Deshacer
`git switch -`
te devuelve a donde estabas.

EN VS CODE

main

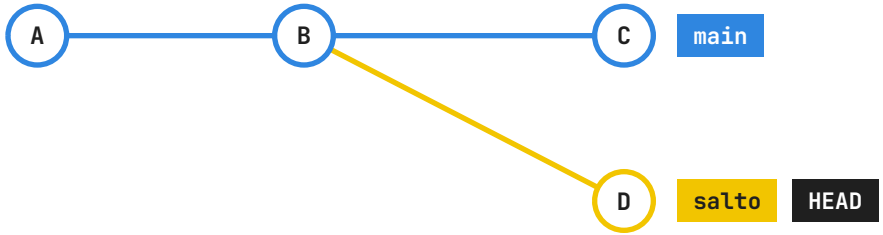
Pincha en tu rama, en la barra de abajo, y elige otra.
Por internet verás `git checkout`, que también vale.

git switch

Trabajo › Stage › Local › Remoto

Cambiar de rama

DESPUÉS HEAD se mueve a `salto` y ahora ves los archivos de `D`.



TERMINAL

```
$ git switch <rama>
```

`-c <rama>`	la crea y te mueve a ella
`-`	vuelve a la rama anterior

Cuidado
Si tienes cambios sin guardar, igual no te deja. Haz commit antes.

Deshacer
`git switch -`
te devuelve a donde estabas.

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

`>Git: Merge...`

3 Elige la rama

`salto`

`origin/main`

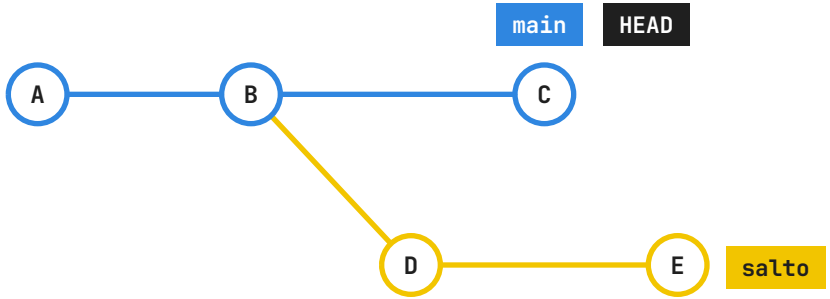
También en Source Control:
⋯ › Branch › Merge...

git merge

Trabajo › Stage › **Local** › Remoto

Unir dos ramas

ANTES Las dos ramas han avanzado por separado. Estás en `main`.



TERMINAL

```
$ git merge <rama>
```

`--no-ff`	crea siempre un commit de merge
`--abort`	cancela un merge con conflictos

⚠ Cuidado
Trae los cambios a la rama en la que estés. Muévete antes.

Deshacer
`git reset --hard ORIG_HEAD` borra lo no commiteado.

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

`>Git: Merge...`

3 Elige la rama

`salto`

`origin/main`

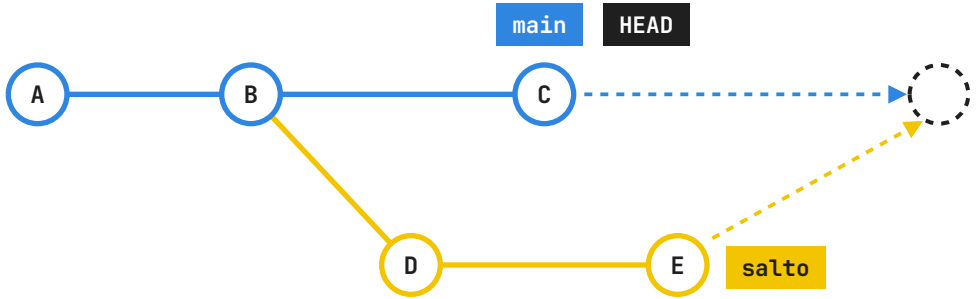
También en Source Control:
⋯ › Branch › Merge...

git merge

Trabajo › Stage › **Local** › Remoto

Unir dos ramas

MERGE `git merge salto` junta lo de `salto` con lo de `main`.



TERMINAL

```
$ git merge <rama>
```

`--no-ff`	crea siempre un commit de merge
`--abort`	cancela un merge con conflictos

⚠ Cuidado
Trae los cambios a la rama en la que estés. Muévete antes.

↺ Deshacer
`git reset --hard ORIG_HEAD` borra lo no commiteado.

EN VS CODE

1 Abre la paleta

`Ctrl` `Shift` `P`

2 Escribe el comando

`>Git: Merge...`

3 Elige la rama

`salto`

`origin/main`

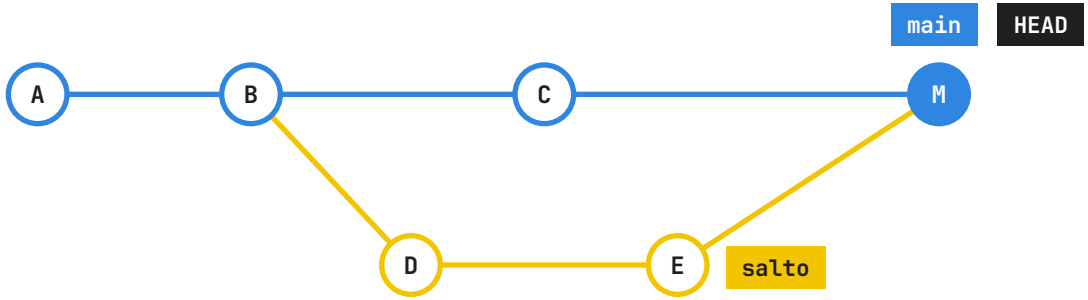
También en Source Control:
⋯ › Branch › Merge...

git merge

Trabajo › Stage › **Local** › Remoto

Unir dos ramas

DESPUÉS Ahora `main` lo tiene todo, y `salto` sigue donde estaba.



TERMINAL

```
$ git merge <rama>
```

`--no-ff`	crea siempre un commit de merge
`--abort`	cancela un merge con conflictos

⚠ Cuidado
Trae los cambios a la rama en la que estés. Muévete antes.

↺ Deshacer
`git reset --hard ORIG_HEAD` borra lo no commiteado.

SOURCE CONTROL

MERGE CHANGES 1

salto.py !

El ! es el conflicto. Pincha en el archivo para abrirlo.

conflicto

Trabajo › Stage › Local › Remoto

Resolver un conflicto

APARECE Las dos ramas han cambiado la misma línea y Git no va a elegir por ti.

```
$ git merge salto

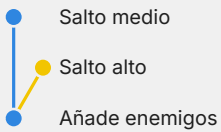
Auto-merging salto.py

CONFLICT (content): Merge conflict in salto.py

Automatic merge failed; fix conflicts and then

commit the result.
```

GRAPH



TERMINAL

$ git add <archivo>

`git commit --no-edit`	termina el merge con su mensaje
`git status`	lista los archivos en conflicto
`git merge --abort`	cancelas y vuelves a como estabas

Cuidado

Borra las marcas <<<<<<<.
Si se quedan, el código peta.

Deshacer

`git merge --abort`
mientras no hayas hecho commit.

SOURCE CONTROL

MERGE CHANGES 1

salto.py !

El ! es el conflicto. Pincha en el archivo para abrirlo.

conflicto

Trabajo › Stage › Local › Remoto

Resolver un conflicto

EN EL ARCHIVO Arriba lo tuyo y abajo lo que llega. VS Code te pone botones para elegir.

salto.py

```
def saltar():
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<<< HEAD  (lo tuyo)
    altura = 12
=======
    altura = 15
>>>>>>> salto  (lo que llega)
```

GRAPH

Salto medio main

Salto alto salto

Añade enemigos

TERMINAL

```
$ git add <archivo>
```

`git commit --no-edit`	termina el merge con su mensaje
`git status`	lista los archivos en conflicto
`git merge --abort`	cancelas y vuelves a como estabas

Cuidado

Borra las marcas <<<<<<<.
Si se quedan, el código peta.

Deshacer

`git merge --abort`
mientras no hayas hecho commit.

SOURCE CONTROL

Continue

Dale al + del archivo y este botón termina el merge.

conflicto

Trabajo › Stage › Local › Remoto

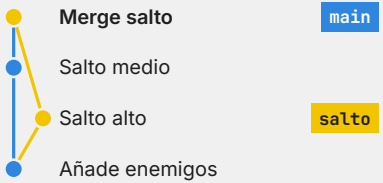
Resolver un conflicto

RESUELTO Deja el archivo como quieras, haz add y termina el merge.

salto.py

```
def saltar():
    altura = 15
    return altura
```

GRAPH



TERMINAL

```
$ git add <archivo>
```

`git commit --no-edit`	termina el merge con su mensaje
`git status`	lista los archivos en conflicto
`git merge --abort`	cancelas y vuelves a como estabas

Cuidado

Borra las marcas <<<<<<<.
Si se quedan, el código peta.

Deshacer

`git merge --abort`
mientras no hayas hecho commit.

ejercicio-2

Ejercicio 2: ramas y conflicto

Unos 10 minutos, en el mismo repo. Vas a liarla a propósito.



1	Crea una rama	`git switch -c experimento`
2	Cambia la línea 1 de `hola.txt` y guarda	`git commit -am "Versión A"`
3	Vuelve a `main`	`git switch main`
4	Cambia la misma línea, con otro texto, y guarda	`git commit -am "Versión B"`
5	Haz el merge y te saltará el conflicto	`git merge experimento`
6	Resuélvelo en VS Code, termina el merge y súbelo	`git add hola.txt; git commit --no-edit; git push`

> **Al terminar** Con `git log --oneline --graph` ves cómo se juntan las dos ramas.





extra

Extra: tu primera pull request

Si nos sobra tiempo. Todos al mismo repo, y las PR van saliendo en directo.



1	Haz fork desde la web	`github.com/guluc3m/actividad-charla-git`
2	Clona tu fork	`git clone <url-de-tu-fork>`
3	Crea `<NIA>.txt` con el mensaje que quieras	
4	Guárdalo y súbelo	`git add .; git commit -m "Añade mi NIA"; git push`
5	Abre la pull request desde la web	

> **Al terminar** Tu PR sale en el repo de todos.





slides

Las diapositivas

Por si quieres seguirlas desde el portátil.



PENDIENTE: falta el enlace de este año

gracias

¡Gracias!

Esto es lo siguiente que tenemos.

Nuevo taller de Git

PRÓXIMAMENTE

¡Estad atentos!

AISC x Google

Inside Google: Ciberseguridad y Datos
10/11 · 14:15 · Aula 2.3.A03

Telegram

Grupo de Telegram del GUL

gul.uc3m.es

`aiscmadrid.com` `gul.uc3m.es`